B2B Travel Booking & Revenue Analytics

PHASE 4 - ANALYTICS DATA MARTS

Complete Beginner-Friendly Documentation

What to do | How to do it | Why to do it | Concepts | SQL | Validation | Analyst thinking

Core Phase 4 idea Convert detailed, FX-normalized transaction data into business-ready summary tables with a clearly defined grain, then reconcile every mart back to the source facts.

Project stack: PostgreSQL / pgAdmin | SQL | Python | Power BI | Excel

Learning mode: beginner-friendly, step-by-step

1. Phase 4 Overview

Phase 4 sits between the trusted FX-normalized facts and the later business analysis / Power BI layer. The goal is not to create new raw facts. The goal is to reshape the same trusted booking and cancellation populations into small, purposeful summary tables that answer recurring business questions quickly and consistently.

What Phase 4 answers

- How is the business performing month by month?
- Is booking, GBV or net revenue growing compared with the previous month?
- Which travel agents generate volume, GBV, revenue and cancellations?
- Which destinations, suppliers and hotels drive commercial performance?
- Why are bookings cancelled and how much refund / fee value is associated with each reason?
- How do we prove that aggregation did not lose bookings or money?
- How do we enforce the intended mart grain with unique indexes?

Phase 4 output tables

Mart	Intended grain	Main use
analytics.monthly_performance	1 row per booking month	Monthly KPIs
analytics.monthly_growth	1 row per booking month	Month-over-month growth
analytics.agent_performance	1 row per agent	Agent performance
analytics.destination_performance	1 row per destination	Destination performance
analytics.supplier_performance	1 row per supplier bucket	Supplier economics
analytics.hotel_performance	1 row per hotel	Hotel performance
analytics.cancellation_analysis	1 row per cancellation reason	Cancellation root-cause summary

Important A data mart is useful only if its grain is explicit. “One row per X” should be written down before the SQL is written.

2. Architecture, Grain and Reconciliation

The Phase 4 pipeline is:

RAW -> STAGING -> CLEAN -> FX-NORMALIZED ANALYTICS -> DATA MARTS -> POWER BI / BUSINESS ANALYSIS

Why not connect Power BI only to the booking fact?

A detailed fact table is essential, but many dashboard questions repeat the same GROUP BY, SUM, COUNT and rate formulas. A mart calculates those business definitions once, at a controlled grain, so downstream tools can use consistent logic.

The reconciliation rule

Grouping changes the number of rows, but it should not change the overall business totals when all source rows are preserved. For example, 119,877 valid booking rows can become 24 monthly rows, but the sum of monthly bookings, GBV and net revenue should equal the source fact totals.

```
Fact table (booking grain)
        |
        | GROUP BY month / agent / destination / supplier / hotel
        v
Business mart (summary grain)
        |
        | SUM summary totals again
        v
Must reconcile to source totals
```

Core formulas reused in Phase 4

Metric	Formula
Average Booking Value (ABV)	Total GBV / Total Bookings
Gross Margin %	Total Net Revenue / Total GBV x 100
Cancellation Rate %	Cancelled Bookings / Total Bookings x 100
MoM Growth %	(Current - Previous) / Previous x 100

Analyst rule For ratio KPIs such as gross margin %, calculate the ratio from aggregated numerators and denominators. Do not average row-level percentages unless that is explicitly the business definition.

3. Lesson 1 - Monthly Performance Mart

Business question: How is the travel business performing month by month?

Grain: one row = one booking month.

What the mart calculates

- Total bookings
- Completed bookings
- Cancelled bookings
- Total GBV in INR
- Total net revenue in INR
- Average booking value
- Gross margin %
- Cancellation rate %

```
CREATE TABLE analytics.monthly_performance AS
SELECT
    DATE_TRUNC('month', booking_dt)::date AS booking_month,
    COUNT(*) AS total_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Completed') AS completed_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Cancelled') AS cancelled_bookings,
    SUM(gross_booking_value_inr) AS total_gbv_inr,
    SUM(net_revenue_inr) AS total_net_revenue_inr,
    ROUND(SUM(gross_booking_value_inr) / NULLIF(COUNT(*), 0), 2)
        AS avg_booking_value_inr,
    ROUND((SUM(net_revenue_inr)
        / NULLIF(SUM(gross_booking_value_inr), 0)) * 100, 2)
        AS gross_margin_pct,
    ROUND((COUNT(*) FILTER (WHERE booking_status = 'Cancelled'))::numeric
        / NULLIF(COUNT(*), 0) * 100, 2)
        AS cancellation_rate_pct
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0
GROUP BY DATE_TRUNC('month', booking_dt)::date
ORDER BY booking_month;
```

Why DATE_TRUNC is used

DATE_TRUNC('month', booking_dt) converts every booking date in the same month to the same month key, such as 2025-06-01. SQL can then aggregate all June 2025 bookings together.

Validation query

```
SELECT
    COUNT(*) AS total_months,
    MIN(booking_month) AS first_month,
    MAX(booking_month) AS last_month
FROM analytics.monthly_performance;
```

Expected from the project date range: about 24 rows, from 2024-01-01 through 2025-12-01. The user did not report this validation output during the lesson, so it remains a checkpoint before final portfolio publication.

4. Lesson 2 - Reconcile the Monthly Mart

A mart is not trusted merely because the SQL runs. We prove that aggregation preserved the source population and money.

Source totals

```
SELECT
    COUNT(*) AS fact_total_bookings,
    SUM(gross_booking_value_inr) AS fact_total_gbv_inr,
    SUM(net_revenue_inr) AS fact_total_net_revenue_inr
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0;
```

Mart totals

```
SELECT
    SUM(total_bookings) AS mart_total_bookings,
    SUM(total_gbv_inr) AS mart_total_gbv_inr,
    SUM(total_net_revenue_inr) AS mart_total_net_revenue_inr
FROM analytics.monthly_performance;
```

Direct difference check

```
SELECT
    (SELECT COUNT(*) FROM analytics.fact_bookings_fx WHERE is_fx_missing = 0)
      - (SELECT SUM(total_bookings) FROM analytics.monthly_performance)
      AS booking_difference,
    (SELECT SUM(gross_booking_value_inr) FROM analytics.fact_bookings_fx WHERE is_fx_missing = 0)
      - (SELECT SUM(total_gbv_inr) FROM analytics.monthly_performance)
      AS gbv_difference,
    (SELECT SUM(net_revenue_inr) FROM analytics.fact_bookings_fx WHERE is_fx_missing = 0)
      - (SELECT SUM(total_net_revenue_inr) FROM analytics.monthly_performance)
      AS revenue_difference;
```

Target result: booking_difference = 0, gbv_difference = 0, revenue_difference = 0. These exact outputs were not reported in the lesson and should be rerun as part of the final Phase 4 checklist.

5. Lesson 3 - Monthly Growth with LAG()

Business question: Is performance growing or shrinking compared with the previous month?

New SQL concept: LAG() is a window function that reads a value from the previous row in an ordered result without collapsing the rows.

```
CREATE TABLE analytics.monthly_growth AS
WITH previous_month_values AS (
    SELECT
        booking_month,
        total_bookings,
        total_gbv_inr,
        total_net_revenue_inr,
        LAG(total_bookings) OVER (ORDER BY booking_month)
            AS previous_month_bookings,
        LAG(total_gbv_inr) OVER (ORDER BY booking_month)
            AS previous_month_gbv_inr,
        LAG(total_net_revenue_inr) OVER (ORDER BY booking_month)
            AS previous_month_net_revenue_inr
    FROM analytics.monthly_performance
)
SELECT
    booking_month,
    total_bookings,
    total_gbv_inr,
    total_net_revenue_inr,
    previous_month_bookings,
    previous_month_gbv_inr,
    previous_month_net_revenue_inr,
    ROUND(((total_bookings - previous_month_bookings)::numeric
        / NULLIF(previous_month_bookings, 0)) * 100, 2)
        AS booking_growth_pct,
    ROUND(((total_gbv_inr - previous_month_gbv_inr)
        / NULLIF(previous_month_gbv_inr, 0)) * 100, 2)
        AS gbv_growth_pct,
    ROUND(((total_net_revenue_inr - previous_month_net_revenue_inr)
        / NULLIF(previous_month_net_revenue_inr, 0)) * 100, 2)
        AS net_revenue_growth_pct
FROM previous_month_values
ORDER BY booking_month;
```

Why the first month is NULL

The first month has no earlier month to compare against. A NULL growth rate is therefore correct, not a data error.

Validation

```
SELECT
    COUNT(*) AS total_months,
    COUNT(*) FILTER (WHERE booking_growth_pct IS NULL)
        AS months_without_booking_growth
FROM analytics.monthly_growth;
```

Expected pattern: total_months approximately 24 and months_without_booking_growth = 1. The lesson output was not reported, so confirm this later.

6. Lesson 4 - Agent Performance Mart

Business question: Which travel agents generate booking volume, GBV, revenue and cancellations?

Grain: one row = one agent.

```
CREATE TABLE analytics.agent_performance AS
SELECT
    agent_id,
    COUNT(*) AS total_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Completed') AS completed_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Cancelled') AS cancelled_bookings,
    SUM(gross_booking_value_inr) AS total_gbv_inr,
    SUM(net_revenue_inr) AS total_net_revenue_inr,
    ROUND(SUM(gross_booking_value_inr) / NULLIF(COUNT(*), 0), 2)
        AS avg_booking_value_inr,
    ROUND((SUM(net_revenue_inr)
        / NULLIF(SUM(gross_booking_value_inr), 0)) * 100, 2)
        AS gross_margin_pct,
    ROUND((COUNT(*) FILTER (WHERE booking_status = 'Cancelled'))::numeric
        / NULLIF(COUNT(*), 0) * 100, 2)
        AS cancellation_rate_pct
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0
GROUP BY agent_id;
```

Grain validation

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT agent_id) AS unique_agents,
    COUNT(*) - COUNT(DISTINCT agent_id) AS duplicate_agents
FROM analytics.agent_performance;
```

The key requirement is duplicate_agents = 0. The exact row count was not reported during the lesson.

Analyst thinking High booking volume alone does not establish strong commercial performance. Compare volume with net revenue, gross margin and cancellation rate.

7. Lesson 5 - Destination Performance Mart

Business question: Which destinations drive demand, GBV, revenue and cancellations?

Grain: one row = one destination.

```
CREATE TABLE analytics.destination_performance AS
SELECT
    destination_id,
    COUNT(*) AS total_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Completed') AS completed_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Cancelled') AS cancelled_bookings,
    SUM(gross_booking_value_inr) AS total_gbv_inr,
    SUM(net_revenue_inr) AS total_net_revenue_inr,
    ROUND(SUM(gross_booking_value_inr) / NULLIF(COUNT(*), 0), 2)
        AS avg_booking_value_inr,
    ROUND((SUM(net_revenue_inr)
        / NULLIF(SUM(gross_booking_value_inr), 0)) * 100, 2)
        AS gross_margin_pct,
    ROUND((COUNT(*) FILTER (WHERE booking_status = 'Cancelled'))::numeric
        / NULLIF(COUNT(*), 0) * 100, 2)
        AS cancellation_rate_pct
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0
GROUP BY destination_id;
```

Validation

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT destination_id) AS unique_destinations,
    COUNT(*) - COUNT(DISTINCT destination_id) AS duplicate_destinations
FROM analytics.destination_performance;
```

Required grain check: duplicate_destinations = 0. The exact result was not reported during the lesson.

8. Lesson 6 - Supplier Performance Mart

Business question: Which suppliers generate booking volume and revenue, and how do supplier cost, margin and cancellations differ?

Grain: one row = one supplier bucket. Missing suppliers were intended to be placed into an UNKNOWN bucket so that bookings and money are not silently dropped.

```
CREATE TABLE analytics.supplier_performance AS
SELECT
    CASE
        WHEN is_supplier_missing = 1 THEN 'UNKNOWN'
        ELSE supplier_id
    END AS supplier_id,
    COUNT(*) AS total_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Completed') AS completed_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Cancelled') AS cancelled_bookings,
    SUM(gross_booking_value_inr) AS total_gbv_inr,
    SUM(supplier_cost_inr) AS total_supplier_cost_inr,
    SUM(net_revenue_inr) AS total_net_revenue_inr,
    ROUND(SUM(gross_booking_value_inr) / NULLIF(COUNT(*), 0), 2)
        AS avg_booking_value_inr,
    ROUND((SUM(net_revenue_inr)
        / NULLIF(SUM(gross_booking_value_inr), 0)) * 100, 2)
        AS gross_margin_pct,
    ROUND((COUNT(*) FILTER (WHERE booking_status = 'Cancelled'))::numeric
        / NULLIF(COUNT(*), 0) * 100, 2)
        AS cancellation_rate_pct
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0
GROUP BY CASE
    WHEN is_supplier_missing = 1 THEN 'UNKNOWN'
    ELSE supplier_id
END;
```

User-confirmed result

Check	Reported result	Interpretation
Total supplier mart rows	161	161 distinct supplier buckets appeared
Unique supplier IDs / buckets	161	Matches total rows
Duplicate supplier buckets	0	Grain is unique
UNKNOWN total bookings	1	Reported, but requires follow-up diagnostic

Open validation issue Earlier clean-layer profiling found 240 missing / blank supplier IDs before the mart. The reported UNKNOWN booking count of 1 does not reconcile cleanly with that history. Recheck NULL, blank-string and is_supplier_missing logic before publishing final portfolio numbers.

```
SELECT
    COUNT(*) FILTER (WHERE supplier_id IS NULL) AS null_supplier_rows,
    COUNT(*) FILTER (WHERE TRIM(COALESCE(supplier_id, '')) = '')
        AS blank_or_null_supplier_rows,
    COUNT(*) FILTER (WHERE is_supplier_missing = 1)
        AS flagged_missing_supplier_rows
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0;
```

9. Lesson 7 - Hotel Performance Mart

Business question: Which hotels generate bookings and revenue, and how do their margins and cancellations compare?

Grain: one row = one hotel.

```
CREATE TABLE analytics.hotel_performance AS
SELECT
    hotel_id,
    COUNT(*) AS total_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Completed') AS completed_bookings,
    COUNT(*) FILTER (WHERE booking_status = 'Cancelled') AS cancelled_bookings,
    SUM(gross_booking_value_inr) AS total_gbv_inr,
    SUM(net_revenue_inr) AS total_net_revenue_inr,
    ROUND(SUM(gross_booking_value_inr) / NULLIF(COUNT(*), 0), 2)
        AS avg_booking_value_inr,
    ROUND((SUM(net_revenue_inr)
        / NULLIF(SUM(gross_booking_value_inr), 0)) * 100, 2)
        AS gross_margin_pct,
    ROUND((COUNT(*) FILTER (WHERE booking_status = 'Cancelled'))::numeric
        / NULLIF(COUNT(*), 0) * 100, 2)
        AS cancellation_rate_pct
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0
GROUP BY hotel_id;
```

Validation

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT hotel_id) AS unique_hotels,
    COUNT(*) - COUNT(DISTINCT hotel_id) AS duplicate_hotels
FROM analytics.hotel_performance;
```

Required result: duplicate_hotels = 0. The exact row count was not reported during the lesson.

10. Lesson 8 - Cancellation Analysis Mart

Business question: Why are bookings cancelled, and how much refund and cancellation-fee value is associated with each reason?

Source: analytics.fact_cancellations_fx. Grain: one row = one cancellation reason.

```
CREATE TABLE analytics.cancellation_analysis AS
SELECT
    cancellation_reason,
    COUNT(*) AS total_cancellations,
    SUM(refund_amount_inr) AS total_refund_amount_inr,
    SUM(cancellation_fee_inr) AS total_cancellation_fee_inr,
    ROUND(SUM(refund_amount_inr) / NULLIF(COUNT(*), 0), 2)
        AS avg_refund_amount_inr,
    ROUND(SUM(cancellation_fee_inr) / NULLIF(COUNT(*), 0), 2)
        AS avg_cancellation_fee_inr
FROM analytics.fact_cancellations_fx
WHERE is_fx_missing = 0
GROUP BY cancellation_reason
ORDER BY total_cancellations DESC;
```

User-confirmed validation

Check	Verified result	Meaning
Total rows	8	Eight cancellation-reason groups
Unique reasons	8	Each row has a distinct reason
Duplicate reasons	0	Correct mart grain
Cancellation difference	0	No cancellation events lost during aggregation

Reconciliation query

```
SELECT
    (SELECT COUNT(*)
       FROM analytics.fact_cancellations_fx
      WHERE is_fx_missing = 0)
    -
    (SELECT SUM(total_cancellations)
       FROM analytics.cancellation_analysis)
    AS cancellation_difference;
```

Result confirmed: cancellation_difference = 0. This is a strong example of reconciliation proving that the mart preserves the source event population.

11. Lesson 9 - Unique Indexes and Final Mart Validation

A unique index both improves access and enforces the intended mart key. If duplicate grain keys exist, PostgreSQL will reject the unique index instead of silently allowing an ambiguous mart.

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_monthly_performance_month
ON analytics.monthly_performance (booking_month);

CREATE UNIQUE INDEX IF NOT EXISTS idx_monthly_growth_month
ON analytics.monthly_growth (booking_month);

CREATE UNIQUE INDEX IF NOT EXISTS idx_agent_performance_agent
ON analytics.agent_performance (agent_id);

CREATE UNIQUE INDEX IF NOT EXISTS idx_destination_performance_destination
ON analytics.destination_performance (destination_id);

CREATE UNIQUE INDEX IF NOT EXISTS idx_supplier_performance_supplier
ON analytics.supplier_performance (supplier_id);

CREATE UNIQUE INDEX IF NOT EXISTS idx_hotel_performance_hotel
ON analytics.hotel_performance (hotel_id);

CREATE UNIQUE INDEX IF NOT EXISTS idx_cancellation_analysis_reason
ON analytics.cancellation_analysis (cancellation_reason);
```

All-mart row-count check

```
SELECT 'monthly_performance' AS mart_name, COUNT(*) AS row_count
FROM analytics.monthly_performance
UNION ALL SELECT 'monthly_growth', COUNT(*) FROM analytics.monthly_growth
UNION ALL SELECT 'agent_performance', COUNT(*) FROM analytics.agent_performance
UNION ALL SELECT 'destination_performance', COUNT(*) FROM analytics.destination_performance
UNION ALL SELECT 'supplier_performance', COUNT(*) FROM analytics.supplier_performance
UNION ALL SELECT 'hotel_performance', COUNT(*) FROM analytics.hotel_performance
UNION ALL SELECT 'cancellation_analysis', COUNT(*) FROM analytics.cancellation_analysis;
```

The user requested the PDF before reporting these final row counts, so this is a pending validation checkpoint.

12. Final Financial Reconciliation

Every booking-based mart should reconcile to the same booking fact population because each mart is simply a different grouping of the same valid bookings.

Source fact totals

```
SELECT
    COUNT(*) AS fact_bookings,
    SUM(gross_booking_value_inr) AS fact_gbv_inr,
    SUM(net_revenue_inr) AS fact_net_revenue_inr
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0;
```

Compare mart totals

```
SELECT 'monthly_performance' AS mart_name,
       SUM(total_bookings), SUM(total_gbv_inr), SUM(total_net_revenue_inr)
FROM analytics.monthly_performance
UNION ALL
SELECT 'agent_performance',
       SUM(total_bookings), SUM(total_gbv_inr), SUM(total_net_revenue_inr)
FROM analytics.agent_performance
UNION ALL
SELECT 'destination_performance',
       SUM(total_bookings), SUM(total_gbv_inr), SUM(total_net_revenue_inr)
FROM analytics.destination_performance
UNION ALL
SELECT 'supplier_performance',
       SUM(total_bookings), SUM(total_gbv_inr), SUM(total_net_revenue_inr)
FROM analytics.supplier_performance
UNION ALL
SELECT 'hotel_performance',
       SUM(total_bookings), SUM(total_gbv_inr), SUM(total_net_revenue_inr)
FROM analytics.hotel_performance;
```

Expected booking population: 119,877 valid FX-normalized bookings. All booking marts should sum back to the same booking count, GBV and net revenue. These final reconciliation outputs were not supplied before this document was requested, so do not publish them as “verified” until rerun.

Why monthly_growth is different

monthly_growth is derived from monthly_performance and contains current / previous month values plus growth rates. It is not used as a second independent financial reconciliation source because previous-month columns intentionally repeat prior values.

13. Phase 4 Validation Status

Area	Status	Evidence / next action
Monthly performance grain	Pending confirmation	Run 24-month / min / max validation
Monthly reconciliation	Pending confirmation	Differences should be 0 / 0 / 0
Monthly growth	Pending confirmation	About 24 rows; first growth should be NULL
Agent mart grain	Pending confirmation	duplicate_agents should be 0
Destination mart grain	Pending confirmation	duplicate_destinations should be 0
Supplier mart grain	Verified	161 rows, 161 unique, 0 duplicate
Supplier missing bucket	Needs investigation	UNKNOWN bookings reported as 1; recheck NULL / blank / flag logic
Hotel mart grain	Pending confirmation	duplicate_hotels should be 0
Cancellation mart grain	Verified	8 rows, 8 unique, 0 duplicate
Cancellation reconciliation	Verified	difference = 0
Unique indexes	Pending confirmation	Create all seven unique indexes
Cross-mart financial reconciliation	Pending confirmation	All booking marts should match 119,877-booking fact totals

Documentation principle “Taught” and “verified” are not the same thing. This PDF records both the complete Phase 4 logic and the exact validation items still waiting for reported outputs.

14. How These Marts Support Business Analysis

Phase 4 is the bridge between technical cleaning and business interpretation. Each mart supports a different management view while keeping KPI definitions consistent.

Business question	Best mart	Typical fields
How are we performing over time?	monthly_performance	bookings, GBV, revenue, margin, cancellation rate
Are we growing month over month?	monthly_growth	booking / GBV / revenue growth %
Which agents drive activity?	agent_performance	bookings, GBV, revenue, cancellations
Which markets drive demand?	destination_performance	bookings, GBV, revenue, margin
How do suppliers affect economics?	supplier_performance	GBV, supplier cost, revenue, margin
Which hotels generate activity?	hotel_performance	bookings, GBV, revenue, cancellations
Why are cancellations happening?	cancellation_analysis	reason counts, refunds, fees

A good dashboard does not need every raw field

Power BI can still connect to detailed facts for drill-through, but the marts provide clean summary layers for recurring visuals and KPI cards. This separation reduces repeated formula logic and makes validation easier.

Examples of analyst questions now possible

- Which months show the strongest or weakest GBV growth?
- Do high-volume agents also generate healthy margin?
- Which destinations combine high demand with high cancellation rates?
- Are certain suppliers associated with high supplier cost relative to GBV?
- Which cancellation reasons account for the largest refund amounts?

15. Key Concepts, Common Mistakes and Interview Talking Points

Key concepts learned

- Data mart: a purpose-built analytical table for a recurring business question.
- Grain: the business meaning of one row in a table.
- Reconciliation: proving summary totals match the trusted source totals.
- Window function: LAG() compares current and previous rows without GROUP BY collapsing them.
- Conditional aggregation: FILTER counts only rows meeting a business condition.
- NULLIF: prevents division-by-zero errors in ratios.
- UNKNOWN dimension bucket: preserves business facts when a dimension reference is missing.
- Unique index: enforces the intended mart key and can improve lookup performance.

Common mistakes to avoid

- Creating a mart without defining its grain first.
- Dropping rows with missing dimension IDs instead of explicitly deciding how to handle them.
- Averaging row-level percentages when the business metric should be computed from aggregate totals.
- Using previous-month values without an ORDER BY inside LAG().
- Assuming a CREATE TABLE succeeded means the numbers are correct.
- Skipping reconciliation after GROUP BY transformations.
- Joining lower-grain payment transactions directly into booking marts and accidentally multiplying booking money.

Interview-ready explanation

> “After cleaning and FX-normalizing the booking, cancellation and payment data, I created business-specific PostgreSQL marts at controlled grains: month, agent, destination, supplier, hotel and cancellation reason. I used conditional aggregation for KPIs, LAG() for month-over-month growth, preserved missing supplier facts with an UNKNOWN bucket, and reconciled marts back to the trusted fact tables before using them downstream in BI.”

16. Quick Reference and Phase 5 Handoff

Reusable SQL patterns

```
-- Conditional count
COUNT(*) FILTER (WHERE condition)

-- Safe ratio
numerator / NULLIF(denominator, 0)

-- Monthly key
DATE_TRUNC('month', some_date)::date

-- Previous row
LAG(metric) OVER (ORDER BY month_key)

-- Uniqueness check
COUNT(*) - COUNT(DISTINCT grain_key)

-- Missing-dimension bucket
CASE WHEN missing_flag = 1 THEN 'UNKNOWN' ELSE dimension_id END
```

Before moving to final portfolio analysis

- Run the pending mart row-count and uniqueness checks.
- Run final cross-mart booking / GBV / net-revenue reconciliation.
- Resolve the supplier missing / UNKNOWN discrepancy.
- Confirm unique indexes were created successfully.
- Record the final verified outputs in your project notes.

Next phase

Phase 5 will use these marts to answer business questions with SQL: growth drivers, profitability patterns, agent / market / supplier performance, cancellation impact and actionable insights. The emphasis shifts from building trusted analytical structures to interpreting them.

Phase 4 takeaway A good analyst does not only calculate KPIs. A good analyst defines the grain, preserves the population, validates the math, and makes every downstream number traceable to a trusted source.